EduX

School Management System

System Architecture

Developer Documentation

Version 1.0
February 7, 2026

Contents

1 Architecture Overview **2**
- 1.1 Introduction . . . 2
- 1.2 Technology Stack . . . 2
- 1.3 Three-Layer Architecture . . . 3

2 Presentation Layer **4**
- 2.1 Overview . . . 4
- 2.2 Directory Structure . . . 4
- 2.3 Screen Architecture . . . 5
- 2.4 Navigation with go_router . . . 5
  - 2.4.1 Route Structure . . . 5

3 Domain Layer **6**
- 3.1 Services . . . 6
  - 3.1.1 Service Responsibilities . . . 6
- 3.2 Providers (Riverpod) . . . 6
  - 3.2.1 Provider Types Used . . . 7
  - 3.2.2 Provider Architecture . . . 7

4 Data Layer **8**
- 4.1 Drift ORM . . . 8
  - 4.1.1 Key Features . . . 8
- 4.2 Repository Pattern . . . 8
  - 4.2.1 Repository Responsibilities . . . 8

5 Data Flow **9**
- 5.1 Read Operation Flow . . . 9
- 5.2 Write Operation Flow . . . 10

6 Module Structure **11**
- 6.1 Feature Modules . . . 11
- 6.2 Module Dependency Graph . . . 11

7 Design Patterns **12**
- 7.1 Patterns Used . . . 12
- 7.2 Error Handling . . . 12

Chapter 1

Architecture Overview

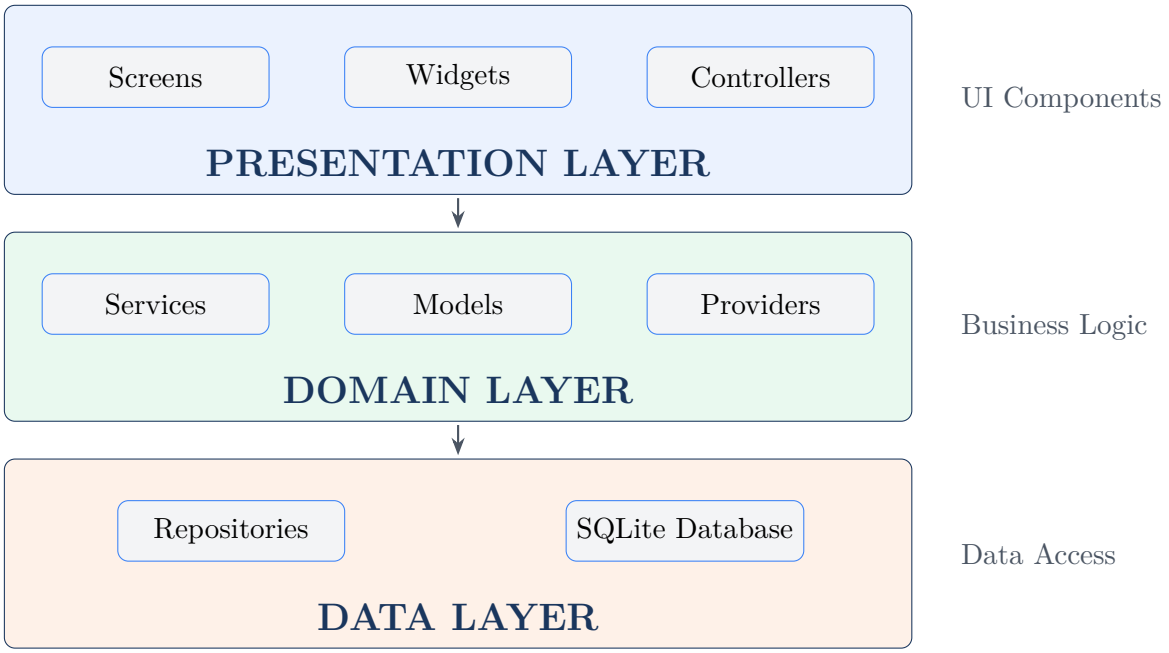
1.1 Introduction

EduX is built using a clean, layered architecture that separates concerns and promotes maintainability. The application follows the Presentation-Domain-Data pattern with Riverpod for state management.

1.2 Technology Stack

Component	Technology
Framework	Flutter 3.x (Desktop - Windows)
Language	Dart 3.x
Database	SQLite via Drift ORM
State Management	Riverpod 2.x
Navigation	go_router
PDF Generation	pdf + printing packages
Excel Support	excel package
Charts	fl_chart

1.3 Three-Layer Architecture



Chapter 2

Presentation Layer

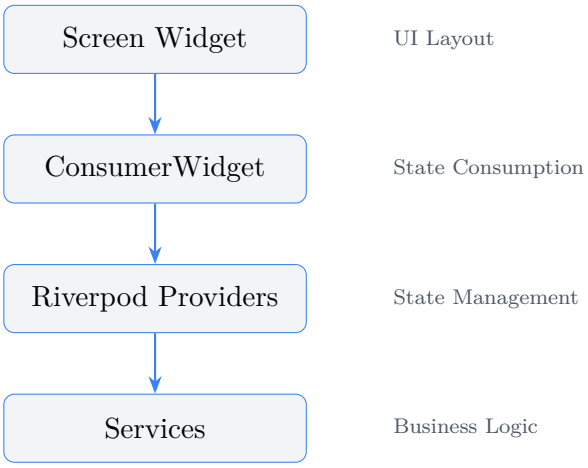
2.1 Overview

The Presentation Layer handles all UI-related code including screens, widgets, and user interaction logic.

2.2 Directory Structure

```
lib/
|-- screens/            # Full-page screens
|   |-- students/       # Student module screens
|   |-- attendance/     # Attendance screens
|   |-- fees/           # Fee management screens
|   |-- exams/          # Examination screens
|   |-- staff/          # Staff management screens
|   |-- academics/      # Academic screens
|   |-- guardians/      # Guardian screens
|
|-- features/           # Feature-specific widgets
|   |-- dashboard/      # Dashboard components
|   |-- settings/       # Settings screens
|   |-- auth/           # Authentication UI
|   |-- shell/          # App shell with navigation
|
|-- core/
    |-- widgets/        # Shared/reusable widgets
    |-- theme/          # Theme configuration
```

2.3 Screen Architecture



2.4 Navigation with go_router

The application uses `go_router` for declarative routing with nested navigation.

2.4.1 Route Structure

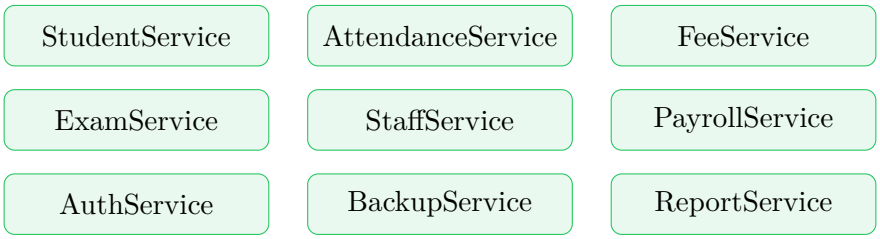
```
/                       # Splash screen
/login                  # Authentication
/setup                  # First-time setup
/app                    # Main app shell
  /app/dashboard        # Dashboard
  /app/students         # Student listing
  /app/students/:id     # Student details
  /app/attendance       # Attendance
  /app/fees             # Fee management
  /app/exams            # Examinations
  /app/staff            # Staff management
  /app/settings         # Settings
```

Chapter 3

Domain Layer

3.1 Services

Services contain business logic and orchestrate data operations.



3.1.1 Service Responsibilities

Service	Responsibilities
StudentService	CRUD operations, enrollment, status management
AttendanceService	Mark attendance, alerts, statistics
FeeService	Fee structure, concessions
InvoiceService	Generate, update invoices
PaymentService	Record payments, receipts
ExamService	Exam CRUD, marks, results
StaffService	Staff CRUD, assignments
PayrollService	Salary calculation, processing
AuthService	Login, logout, password management
BackupService	Create, restore backups
ReportService	Generate various reports

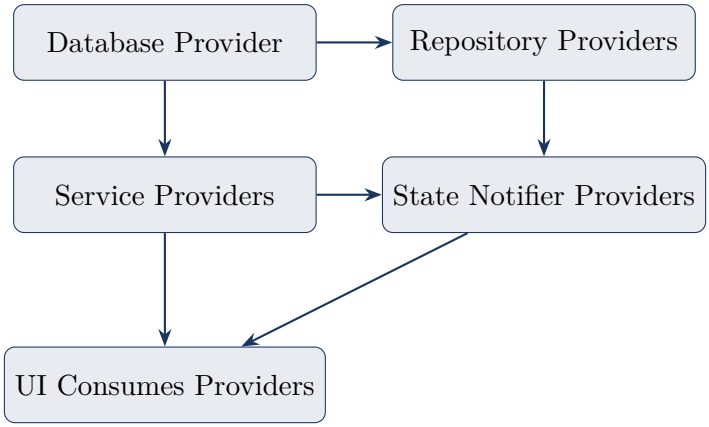
3.2 Providers (Riverpod)

Providers manage application state and expose data to the UI.

3.2.1 Provider Types Used

Provider Type	Use Case
Provider	Simple computed values, services
StateProvider	Simple mutable state
FutureProvider	Async data fetching
StreamProvider	Real-time data streams
StateNotifierProvider	Complex state with mutations

3.2.2 Provider Architecture



Chapter 4

Data Layer

4.1 Drift ORM

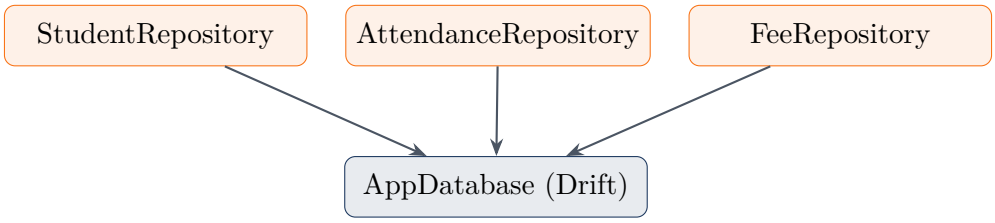
Drift provides type-safe database access with code generation.

4.1.1 Key Features

- Type-safe queries with Dart
- Code generation for tables and DAOs
- Migration support
- Transaction handling
- Stream-based queries for reactive UI

4.2 Repository Pattern

Repositories abstract data source details from the domain layer.



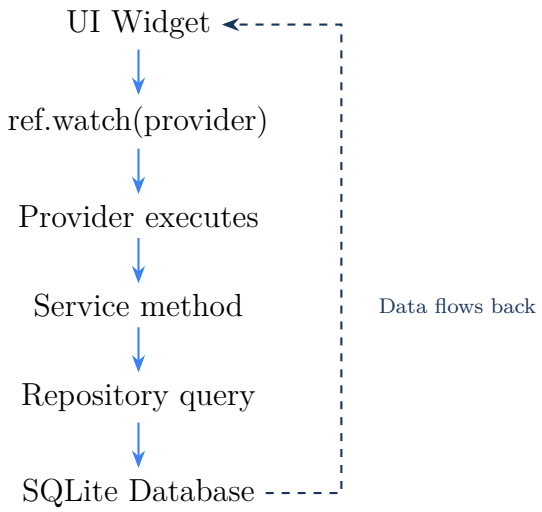
4.2.1 Repository Responsibilities

- CRUD operations on database tables
- Complex queries with joins
- Data transformation
- Caching strategies

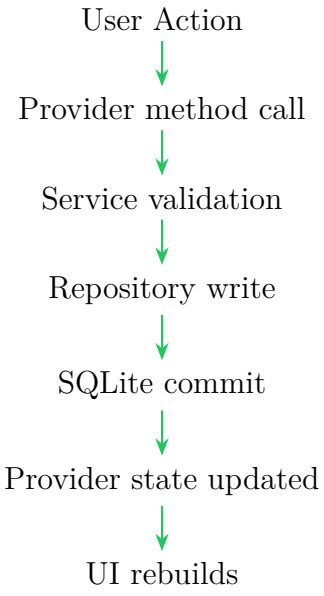
Chapter 5

Data Flow

5.1 Read Operation Flow



5.2 Write Operation Flow



Chapter 6

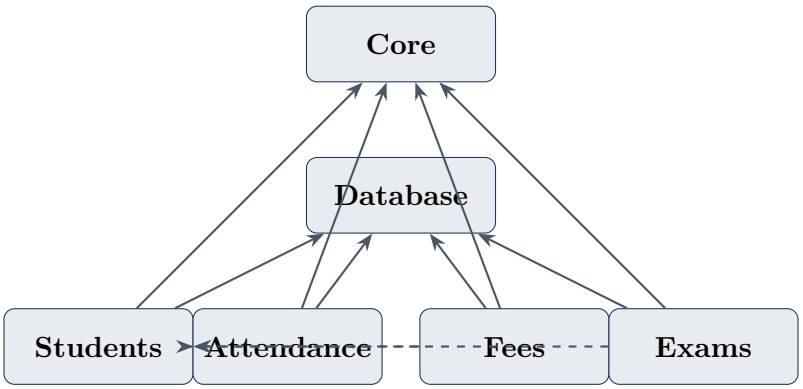
Module Structure

6.1 Feature Modules

Each feature follows a consistent structure:

```
feature_name/
|-- screens/            # Feature screens
|-- widgets/            # Feature-specific widgets
|-- providers/          # Feature providers (optional)
|-- models/             # Feature models (if not shared)
```

6.2 Module Dependency Graph



Chapter 7

Design Patterns

7.1 Patterns Used

Pattern	Application
Repository	Abstracting data access from business logic
Service Layer	Encapsulating business logic
Provider/Observer	Reactive state management with Riverpod
Factory	Creating complex objects (PDF reports)
Singleton	Database instance
Dependency Injection	Via Riverpod providers

7.2 Error Handling

```
// Service layer handles all errors
Future<Result<Student>> createStudent(StudentData data) async {
  try {
    final student = await _repository.insert(data);
    await _activityLog.log(action: 'create', entity: student);
    return Result.success(student);
  } catch (e) {
    return Result.failure(AppError.fromException(e));
  }
}
```